

VAYU — Architecture & Code Walkthrough

Satellite-Derived Surface AQI & HCHO Hotspot Detection over India *Full end-to-end technical analysis — backend logic to frontend.*

0. One unified repository

Everything now lives in a **single repository**: `aksh08022006/vayu-aqi-hcho` (`main`). The Next.js web app sits at the repo root (`app/`, `components/`, `lib/`, `public/`) and the Python research pipeline lives alongside it (`src/`, `pipelines/`, `config/`, `docs/`, `tests/`).

History note. This used to be split across two repos — `cb-ild` held the Python (59 files) while `vayu-aqi-hcho` held only the frontend, which is why the code looked "missing." They have been consolidated. The pre-merge frontend history is preserved on the remote branch `backup/pre-merge`.

What it contains	Where
Next.js frontend (deploys from root)	<code>app/</code> <code>components/</code> <code>lib/</code> <code>public/</code> + <code>package.json</code> , <code>next.config.ts</code>
Python library + pipelines (59 files)	<code>src/isro_aqi/</code> , <code>pipelines/</code>
Config, docs, tests, outputs	<code>config/</code> , <code>docs/</code> , <code>tests/</code> , <code>outputs/</code>

1. What the project actually is

Two research objectives over India, fused into one codebase:

1. **Surface AQI** — predict ground-level pollutant concentrations (PM2.5, PM10, NO₂, SO₂, O₃, CO) from satellite + reanalysis data, then convert them to a daily Air Quality Index map.
2. **HCHO hotspots** — detect formaldehyde hotspots from TROPOMI, attribute them to sources (crop burning / industry / urban / biogenic), and trace upwind atmospheric transport.

It ships as three parts:

- **Python library** (`src/isro_aqi/`) — does the science.
- **Pipeline scripts** (`pipelines/`) — orchestrate runs end-to-end.
- **Next.js scrollytelling site** (`web/`) — visualizes the results.

There are **two data realities** living side by side: a **synthetic demo path** (runs with zero credentials) and a **real satellite path** (validated against CPCB/OpenAQ ground stations).

2. Where the "working of the model" resides

The phrase "the model" maps to **three different things**, deliberately layered:

What	File	Role	Actually used?
RF / XGBoost baselines	<code>src/isro_aqi/models/baselines.py</code>	One regressor per pollutant	RF: yes — the real predictor ; XGB: floor only
CNN	<code>src/isro_aqi/models/cnn.py</code>	Spatial encoder (3 conv blocks → 128-d)	Only <i>inside</i> the CNN-LSTM
CNN-LSTM	<code>src/isro_aqi/models/cnn_lstm.py</code>	The ISRO-"recommended" learner	Trained + scored, not on the inference path
Hybrid (regression-kriging)	<code>src/isro_aqi/models/hybrid.py</code>	$C(s,t) = \mu(\text{trend}) + v(\text{kriged residual})$	Deployed model in the demo (<code>models/hybrid.joblib</code>)
AQI engine	<code>src/isro_aqi/aqi/engine.py</code>	Concentrations → CPCB AQI + entropy RAPI	Always the final stage

Bottom line: the genuinely *operational* predictor is a **Random Forest** — either as the μ trend term inside the hybrid (demo path, `run_demo.py`) or as a bare per-target `RandomForestRegressor(n_estimators=300, min_samples_leaf=2)` on real data (`run_real.py:391`). The **CNN-LSTM is the headline/aspirational model**: fully implemented and trainable (`models/cnn_lstm_demo.pt` exists), reported in validation tables, but **not wired into map generation**. `pipelines/04_train.py:54–64` even leaves its training as a documented stub.

Whatever produces the six concentration grids, the **last mile is always deterministic** —

`AQIEngine.compute_grid()` (`aqi/engine.py:191`) applies the official CPCB piecewise-linear sub-index breakpoints, takes the **max rule** for AQI, and additionally computes the project's novel entropy index **RAPI** and the **divergence** map ($\text{RAPI} - \text{CPCB}$).

3. The model files, in detail

`models/baselines.py` — `_PerTargetModel` trains one independent regressor per target (clean per-pollutant tuning + SHAP). `RandomForestModel` = 300 trees, unbounded depth; `XGBoostModel` = 600 trees, depth 8, lr 0.05, `tree_method="hist"`. The `metrics()` here (`{r2, rmse, mae, n}`, NaN-masked) is the **canonical scorer reused by every other module**.

`models/cnn.py` — `PollutantCNN`: `Conv(C→32→64→128)` with BN/ReLU/MaxPool → `AdaptiveAvgPool2d(1)` → 128-d vector → head. Its `.embed()` (`cnn.py:33`) is the seam the LSTM plugs into.

`models/cnn_lstm.py` — `PollutantCNNLSTM`: input `(B, T, C, P, P)` → shared CNN per daily patch → `(B, T, 128)` → 2-layer LSTM → last timestep → head → `(B, n_targets)`. Defaults: patch 15×15, 7-day look-back. Predicts day-T concentration at the patch center from a week of context.

`models/hybrid.py` — regression-kriging (README "Change 3"). `fit()` trains the RF trend, computes residuals, takes the **per-station mean residual**, and fits `ResidualKriging` — a Gaussian-kernel simple kriging ($w_i = \exp(-d^2/2L^2)$, $L=0.6^\circ$). The crux is the `+reg` damping (`hybrid.py:50`): far from any monitor, weights → 0, the residual → 0, and it **falls back to the pure trend** instead of extrapolating.

`INDIA_BENCHMARK_R2` (pm25=0.86, no2=0.83, ...) holds the literature skill targets.

`models/dataset.py` — tensorizes the gridded stack into `(T, C, P, P)` patches. `Standardizer` per-channel z-scoring; targets z-scored so a large-magnitude pollutant (PM10) doesn't dominate the multi-target MSE. Edge patches reflect-pad; short windows left-pad by repeating the first frame.

`models/train.py` — Adam + early-stopping loop. `masked_mse()` (`train.py:37`) is the key trick: stations reporting only some pollutants contribute **no gradient** on the missing ones. Holds the three Wang-2023 CV splitters: `temporal_split` (held-out years), `spatial_blocks` (0.5° blocks, none split across train/val), and random.

4. The AQI engine (`aqi/engine.py`) — the deterministic core

The most thoroughly unit-tested part of the codebase.

- **Sub-index** (`engine.py:29`): piecewise-linear interpolation between CPCB-2014 breakpoints, $I_p = (I_{hi}-I_{lo})/(C_{hi}-C_{lo}) \cdot (C_p-C_{lo}) + I_{lo}$, capped at 500.
- **Overall AQI** (`engine.py:66`): **max** of the sub-indices, valid only if ≥ 3 pollutants are present *and* PM2.5 or PM10 is among them. Returns AQI + dominant pollutant + category/color.
- **RAPI** (`engine.py:146`) — the project's USP, an entropy-weighted aggregation (Lu 2011): $RAPI = \max(I) \cdot (1 + (\text{mean}(I)/\max(I)) \cdot H)$ where H is normalized Shannon entropy of the sub-index distribution. $RAPI \geq CPCB$ always; equals CPCB when one pollutant dominates and rises when several co-elevate.
- `compute_grid()` returns `{cpcb, rapi, dominant, divergence}` — the dual atlas (official + entropy + the divergence map that is the headline novelty).

Breakpoints/colors come from `config/aqi_breakpoints.yaml` (official 6-band CPCB ramp Good→Severe).

5. Backend — the data layers (bottom-up)

Config (`src/isro_aqi/config.py` + `config/*.yaml`)

`load_config()` merges four YAMLS into a typed pydantic `Config`: - `config.yaml` — GEE project `vayu-500014`, India bbox `[68, 6.5, 97.5, 37.5]`, 0.01° (~1 km) grid, date window, model hyperparameters, validation schemes. - `datasets.yaml` — the registry of every satellite asset ID (table below). -

`regions.yaml` — bounding boxes for HCHO attribution (Indo-Gangetic crop-burning belt, NE forest fires, 7 urban zones, industrial corridors, transport receptor cities). - `aqi_breakpoints.yaml` — CPCB sub-index breakpoints + colors.

Ingestion (`src/isro_aqi/ingestion/`)

File	Source / Asset ID
<code>sentinel5p.py</code>	TROPOMI <code>COPERNICUS/S5P/0FFL/L3_{NO2,S02,C0,03,HCHO}</code> (HCHO extra-masked by <code>cloud_fraction</code>)
<code>era5.py</code>	ECMWF/ERA5_LAND/DAILY_AGGR (temp, wind u/v, RH, pressure, precip, solar) + BLH via Copernicus CDS
<code>modis_fire.py</code>	MOD14A1 (FRP), MCD64A1 (burned area), MOD13A2 (EVI)
<code>viirs_fire.py</code>	NASA FIRMS API (375 m fires) + GEE <code>FIRMS</code> fallback
<code>srtm.py</code>	USGS/SRTMGL1_003 → elevation/slope/aspect
<code>worldcover.py</code>	ESA/WorldCover/v200 → 11 fractional land-cover bands
<code>insat_aod.py</code>	INSAT-3D <code>3DIMG_L2B_AOD</code> via ISRO MOSDAC (stub; MAIAC <code>MCD19A2</code> is the working substitute)
<code>cpcb.py</code>	CPCB ground-station CSVs → daily means (24-h for PM/NO ₂ /SO ₂ , 8-h rolling max for O ₃ /CO) — ground truth
<code>gee_auth.py</code>	Earth Engine init (service-account or interactive), AOI geometry, <code>export_image</code> to GCS/Drive

Preprocessing (`src/isro_aqi/preprocessing/`)

- `assemble.py` — co-registers all exported rasters + INSAT granules onto the 1 km grid into one `(time, lat, lon)` xarray cube.
- `qa_filter.py` — physical-range clipping + 5 σ outlier masking.
- `gapfill_aod.py` — **Change 1:** RF gap-fills ~41%-cloud-missing AOD using only gap-free covariates, validated with *clustered-holdout* CV (random holdout would lie).
- `calibrate_no2.py` — **Change 2:** RF maps TROPOMI NO₂ column → surface NO₂ (raw column underestimates ~4 $\times$).
- `collocate.py` — samples gridded predictors at station coords (`sample_at_stations`) and joins CPCB targets (`join_targets`) → the supervised training table.
- `regrid.py` , `temporal.py` — resampling + Indian IMD-season binning.

Features (`features/engineering.py`)

`add_engineered_features()` adds `fnr = hcho/(no2+ $\epsilon$ )` (ozone-regime indicator), cyclical `doy_sin/doy_cos` , interactions (`aod_blh` , `photo_index`), and optional temporal lags.

Database (`database/schema.py` + `build_db.py`)

One record = one `(date, lat, lon)` cell: **33 predictors + 6 `_obs` targets**. Builds two Hive-partitioned (year/month) parquet products: the **training table** (rows with targets) and the giant **inference grid** (predictors only, ~50–100 M rows).

Synthetic generator (`synthetic.py`)

Produces a physically-plausible India with documented response functions (e.g. `pm25 = 12 + 110·aod + 55·fire + ...`) **plus a hidden per-station emission factor** that is *not* a predictor — so a model can only fit it by memorizing `lat/lon` . This is what makes spatial CV honestly degrade vs random CV, demonstrating the leakage effect.

6. HCHO analysis (`src/isro_aqi/hcho/`) + visualization (`viz/`)

A clean **detect** → **cluster** → **attribute** → **transport** pipeline:

- **phv.py** — "Percentage Higher than Vicinity" (Dong 2026): `PHV = cell / mean(8 Moore neighbors)`. A cell is a hotspot only if it passes **both** a relative test ($PHV > 1$) and an absolute threshold ($\geq 1e16$ molec/cm²); optional change-detection mutation step.
- **getis_ord.py** — Getis-Ord **Gi*** spatial cluster statistic via PySAL/esda `G_Local` (binary 0.05° distance-band weights, 999-permutation p-values), with **Benjamini-Hochberg FDR** correction across the whole map. *Gi** is a z-score; positive significant = hotspot.
- **source_attribution.py** — `connected_clusters` (scipy connected components, a DBSCAN substitute) reduces masks to centroids; `attribute()` runs a rule-based geography + FRP-fire-gate
- EVI + season decision tree → urban / industrial / agri_burning / forest_fire / biogenic / other.
- **transport.py** — kinematic back-trajectory through ERA5 winds (spherical forward-Euler, single level) + vectorized-haversine `fires_along_path` to confirm "Punjab fires → Delhi HCHO" links. HYSPLIT hook stubbed for production.

Visualization — `viz/maps.py` renders India maps (Cartopy-or-fallback) with the official CPCB color ramp; `viz/figures.py` makes hexbin obs-vs-pred validation scatters, SHAP importance bars, and HCHO–O₃ panels. `utils/geo.py` holds the `Grid`, the Moore-neighborhood engine behind PHV, and haversine distance.

7. Pipeline orchestration (`pipelines/` + `Makefile`)

Two tracks: a **numbered phase pipeline** (`01–07` , partly stubbed) and three **self-contained end-to-end entry points** that actually produce all artifacts today.

Numbered (designed flow, partly manual/stub): `01_ingest` (functional, async GEE exports) → `02_preprocess` (functional) → `03_build_database` (functional) → `04_train` (RF/XGB work; CNN-LSTM is a stub) → `05/06/07_*` (**scaffold stubs** — real impl is in the entry-point scripts).

The three real entry points:

1. **run_demo.py** (`make demo` , no credentials) — synthetic ingest → AOD gapfill → collocate/features → NO₂ calibration → **fit & save HybridModel** → 3-scheme CV → train CNN-LSTM → dual AQI atlas → HCHO PHV/Gi*/attribution → transport. Writes `models/` , `outputs/maps` , `demo_summary.md` .
2. **run_real.py** (`OPENAQ_API_KEY=... make real`) — the production real-data run: - **Ground truth:** OpenAQ v3 daily (free, no captcha — replaces the captcha-gated CPCB portal), pulled in parallel from the keyless S3 archive. - **Predictors:** daily TROPOMI/ERA5/MAIAC sampled at *station points* via GEE `getRegion` , snapped to nearest station with a KD-tree. - **Dual CV** (`run_real.py:395`): **random 5-fold** (held-out *days* at known stations — literature-comparable interpolation) **and spatial 2°-block 5-fold** (held-out *regions* — hard extrapolation to unmonitored sites). The gap between them is the honesty signal. - Refit RF per target → predict seasonal grid → `compute_grid` → ocean-mask via `india.geojson` → write **1 real validated frame** to `web/public/data/aqi_frames.json` . Validation → `outputs/real_validation.json` .

3. `fetch_real_web.py` (`make fetch-web`) — replaces the **observation** layers with real 2021 TROPOMI/MAIAC/MODIS + a real ERA5 back-trajectory → `gas_grids/hcho_grid/hotspots/fires/trajectory.json` . `export_web.py` is the demo equivalent (reads `hybrid.joblib` , writes 8 AQI frames).

`check_ingest.py` is a readiness "doctor" — checks Python packages, GEE/CDS/FIRMS credentials, config, and manual inputs before a real run.

8. Real validation results (`outputs/real_validation.json`)

~158 stations, ~4,300 station-days, Oct–Dec 2025:

Pollutant	Random-CV R ²	Spatial-CV R ²
PM2.5	0.53	0.03
PM10	0.58	0.02
NO ₂	0.71	−0.15
O ₃	0.66	—
SO ₂	0.46	−0.96
CO	0.69	0.19

The large random→spatial drop is the spatial-autocorrelation leakage the entire validation framework is built to expose. Random CV measures *interpolation* skill at known stations; spatial CV measures *extrapolation* to unmonitored regions — the honest, harder number.

9. Frontend (`web/` — Next.js 16 + React 19)

A **multi-page scrollytelling site** ("VAYU"), not one long page. Stack: **deck.gl 9 + MapLibre** (no Mapbox token), **Anime.js v4** (not GSAP), **Lenis** smooth scroll, **Tailwind v4**.

- **Routing** (`lib/chapters.ts`): `/ hub` → `/problem` → `/method` → `/aqi` → `/hcho` → `/model` → `/impact` . Each route composes sections from `components/sections.tsx` .
- **DeckMap.tsx** is the map engine — the centerpiece. Instead of thousands of colored squares, it rasterizes gridded data into a **value image shown through a deck.gl BitmapLayer with GPU bilinear filtering + NaN-aware Gaussian blur**, producing smooth IQAir/Windy-style atmospheric gradients, with an invisible `PolygonLayer` for hover hit-testing. Per mode: `aqi` (CPCB/RAPI field from `aqi_frames.json`), `gas` (per-gas ramps from `gas_grids.json`), `hotspots` (HCHO field + colored `ScatterplotLayer` by source + clickable intelligence cards), `transport` (HCHO field + fires + `PathLayer` trajectory).
- **Data loading**: each `DeckMap` lazily `fetch()` es only the `/data/*.json` its mode needs; `india.geojson` loads declaratively as the MapLibre boundary + ocean mask.
- **Synthetic visual layer**: `IndiaField.tsx` (Canvas 2D particle field), `Preloader.tsx` , `Pipeline.tsx` (animated SVG data-flow graph) use `lib/india.ts` 's synthetic `intensityAt()` field, distinct from the real `/data` JSON in `DeckMap`.

10. End-to-end data flow (DAG)

```
config/*.yaml
→ ingestion (GEE / CDS / FIRMS / CPCB) [src/isro_aqi/ingestion]
→ assemble + QA + AOD gapfill + NO2 calib [src/isro_aqi/preprocessing]
→ collocate at stations → training table (parquet)
→ feature engineering [features/engineering.py]
→ MODEL: RF trend (+kriging) / CNN-LSTM [src/isro_aqi/models] ← "the model"
→ AQIEngine.compute_grid (CPCB + RAPI) [aqi/engine.py]
→ HCHO: PHV / Gi* → attribute → transport [src/isro_aqi/hcho]
→ export JSON [pipelines/export_web | fetch_real_web | run_real]
→ web/public/data/*.json
→ DeckMap (BitmapLayer fields) + sections [web/]
```

11. The data contract ([web/public/data/](#))

The seven files are the bridge between backend and frontend:

File	Shape	Written by	Read by
aqi_frames.json	{key, frames:[{date, cells: [[lon,lat,aqi,rapil]]}]}	run_real.py (1 real frame) / export_web.py (8 demo frames)	DeckMap aqi
gas_grids.json	{gases, cells: [{lon,lat,aod,no2,...}]}	fetch_real_web.py / export_web.py	DeckMap gas
hcho_grid.json	[[lon,lat,value]]	same	DeckMap hotspots / transport
hotspots.json	[[lon,lat,source,detail,frp,n]]	same	DeckMap hotspots
fires.json	[[lon,lat,frp]]	same	DeckMap transport
trajectory.json	[[lon,lat]]	same	DeckMap transport
india.geojson	FeatureCollection MultiPolygon	static (immutable boundary / ocean mask)	MapLibre + all writers

12. Honest caveats (worth fixing)

- **The CNN-LSTM is showcased but not deployed** — RF (bare or as hybrid trend) generates every map. [04_train.py](#) 's deep branch and [05/06/07_*.py](#) are stubs.
- **Two accuracy stories coexist on the site:** [model/page.tsx](#) shows the *real* $R^2 \approx 0.53\text{--}0.71$, while the older [Results](#) component in [sections.tsx](#) still shows synthetic $R^2 \approx 0.86/0.93$.
- **The AQI timelapse is over-built:** the UI expects 8 frames ([NFRAMES=8](#)), but [run_real.py](#) writes only 1 real seasonal frame, so the scrubber is currently cosmetic against real data.

- **Mixed time windows on the live site:** AQI layer is real 2025 (`run_real.py`); observation layers are real 2021 (`fetch_real_web.py`).
- `hotspots.json` contains literal `NaN` (invalid JSON) → that fetch can throw and degrade the hotspots layer.
- `ChapterRail.tsx` is dead code (a leftover single-page scroll-spy); live nav is `ChapterNav.tsx` .
- **INSAT-3D AOD ingestion is a `NotImplementedError` stub** — MAIAC (`MCD19A2`) is the working substitute despite the "INSAT-3D" branding.

13. File index (where to look)

Concern	Path
Predictive models	<code>src/isro_aqi/models/{baselines,cnn,cnn_lstm,hybrid,dataset,train}.py</code>
AQI computation	<code>src/isro_aqi/aqi/engine.py</code>
HCHO analysis	<code>src/isro_aqi/hcho/{phv,getis_ord,source_attribution,transport}.py</code>
Data ingestion	<code>src/isro_aqi/ingestion/*.py</code>
Preprocessing	<code>src/isro_aqi/preprocessing/*.py</code>
Feature engineering	<code>src/isro_aqi/features/engineering.py</code>
Database schema/build	<code>src/isro_aqi/database/{schema,build_db}.py</code>
Config (typed)	<code>src/isro_aqi/config.py</code> + <code>config/*.yaml</code>
Synthetic data	<code>src/isro_aqi/synthetic.py</code>
Visualization	<code>src/isro_aqi/viz/{maps,figures}.py</code>
Run end-to-end	<code>pipelines/{run_demo,run_real,fetch_real_web,export_web}.py</code> , <code>Makefile</code>
Frontend	<code>web/app/</code> , <code>web/components/</code> , <code>web/lib/</code> , <code>web/public/data/</code>

Generated as a code walkthrough of the VAYU / ISRO Surface-AQI + HCHO project.